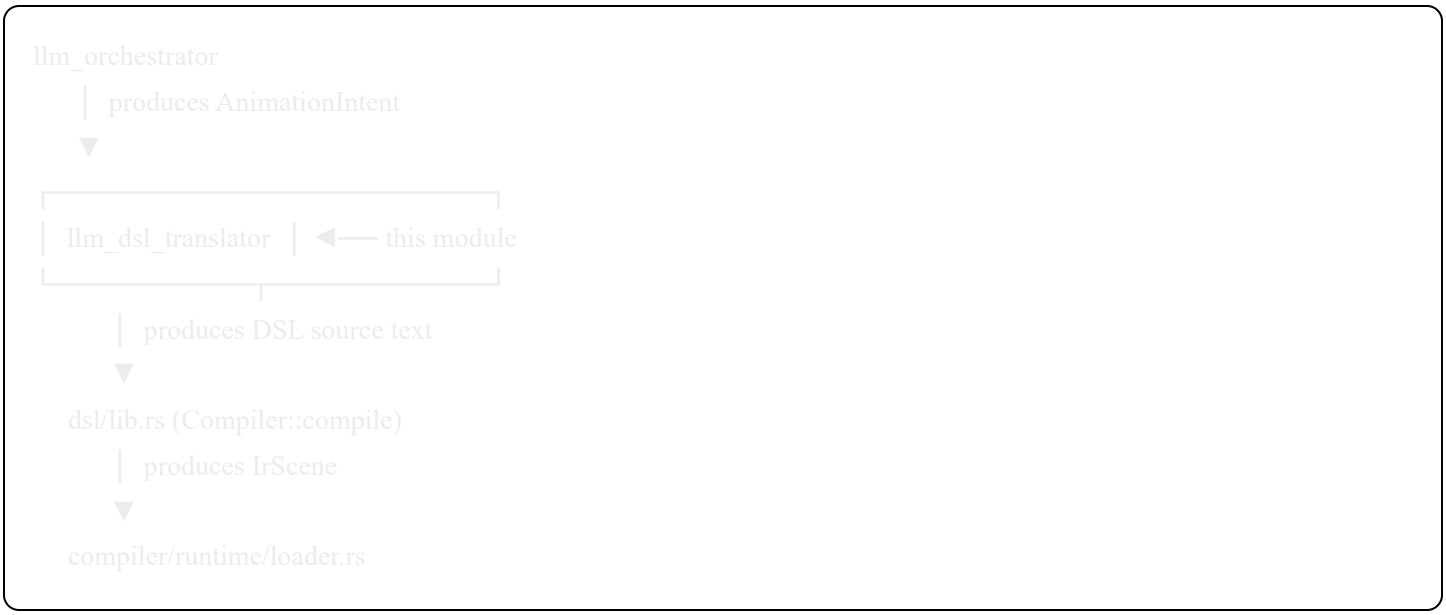
Module: `llm_dsl_translator/`

LLM → DSL Translator

Position in Architecture



The Translator is a pure conversion layer. It receives a structured `AnimationIntent` from the `LLMOrchestrator` and emits a DSL source string that the existing `dsl/lib.rs` compiler can compile without errors. It has no LLM awareness — it cannot call the LLM. On validation failure it returns structured errors that the `FeedbackLoop` in the orchestrator uses to construct a correction prompt.

Directory Layout

```
llm_dsl_translator/
├── mod.rs                  # Public API, Translator struct
├── intent.rs               # AnimationIntent (mirrored from orchestrator, local copy)
├── entity_registry.rs      # EntityRoleRegistry — role name → DSL entity template
├── motion_verb_map.rs      # MotionVerbMap — verb → DSL motion type + parameters
├── dsl_builder.rs          # DslBuilder — programmatic DSL source generator
├── translation_validator.rs # TranslationValidator — validates intent + compiled output
└── id_generator.rs         # IdGenerator — deterministic unique ID generation
```

File: `llm_dsl_translator/intent.rs`

Purpose

Local copy of `AnimationIntent` types. The orchestrator's `animation_intent.rs` and this file define the same types because the orchestrator and translator are separate modules that must not create a circular dependency. The orchestrator depends on the translator (for `TranslationValidator`), so the translator cannot import from the orchestrator. Both modules define compatible structs; the server converts between them using `From` implementations.

What it contains

Same struct definitions as `llm_orchestrator/animation_intent.rs`: `AnimationIntent`, `EntityIntent`, `MotionIntent`, `AnnotationIntent`, `HighlightIntent`, `CameraIntent`.

Additional type: `TranslationError` — returned when translation fails:

- `UnknownEntityRole { role: String, similar: Vec<String> }` — role not in registry; similar names suggest a fix.
- `UnknownMotionVerb { verb: String, similar: Vec<String> }`
- `InvalidMotionParameter { verb: String, param: String, expected_type: String, got: String }`
- `MissingRequiredParameter { verb: String, param: String }`
- `InvalidEntityId { id: String, reason: String }` — e.g., contains invalid DSL identifier characters.
- `DslCompileError { errors: Vec<String> }` — the generated DSL failed the existing compiler's validation pipeline. Errors are the formatted `DslError` messages from `dsl/errors.rs`.
- `TimingConflict { motion_id_1: String, motion_id_2: String }` — two motions conflict on the same entity at the same time.

File: `llm_dsl_translator/entity_registry.rs`

Purpose

`EntityRoleRegistry` maps semantic role names (what the LLM uses to refer to objects) to concrete DSL entity templates (geometry type, scale, material, and default component layout). This is what allows the LLM to say "a radius_vector" and get a properly configured arrow entity in the DSL.

What it contains

`EntityTemplate` struct:

- `role: String` — the role name, e.g. `"unit_circle"`, `"radius_vector"`, `"angle_arc"`, `"x_axis_projection"`, `"y_axis_projection"`, `"coordinate_axes"`, `"function_graph"`, `"tangent_line"`, `"secant_line"`,

`"area_under_curve"`, `"matrix_grid"`, `"eigenvector_arrow"`, `"number_line"`, `"point_on_curve"`, `"normal_vector"`, `"gradient_arrow"`.

- `dsl_kind: String` — the DSL `entity.kind` value: `"solid"`, `"curve"`, `"surface"`, `"field"`.
- `geometry_primitive: String` — the DSL `geometry.primitive` value: `"circle"`, `"sphere"`, `"cylinder"` (used as an arrow), `"arc"`, `"plane"`, `"line"`, etc.
- `default_scale: [f64; 3]`
- `default_material: MaterialTemplate` — color and opacity.
- `default_components: Vec<ComponentTemplate>` — the component blocks to include.
- `description: String` — plain-English description shown to the LLM in its system prompt and to developers.

`MaterialTemplate` struct:

- `color_hex: String`
- `opacity: f64`
- `emissive: bool` — true for objects that should glow (highlight-capable).

`ComponentTemplate` struct:

- `component_type: String` — e.g. `"transform"`, `"geometry"`, `"physical"`, `"kinematic"`.
- `fields: HashMap<String, serde_json::Value>` — default field values.

`EntityRoleRegistry` struct:

- `templates: HashMap<String, EntityTemplate>` — keyed by role name.

Methods:

- `EntityRoleRegistry::builtin() -> Self` — constructs the registry with all pre-defined roles.
- `register(&mut self, template: EntityTemplate)` — adds a new role (extension point).
- `get(&self, role: &str) -> Option<&EntityTemplate>`
- `fuzzy_match(&self, role: &str) -> Vec<&str>` — returns role names with edit distance ≤ 2 from the input. Used by `TranslationError::UnknownEntityRole.similar` to suggest corrections to the LLM.
- `as_json(&self) -> serde_json::Value` — serializes the registry for embedding in LLM prompts.

Builtin roles (minimum set)

The registry must include at minimum:

- `unit_circle` — a circle of radius 1 centered at origin
- `radius_vector` — an arrow from origin to a point on the circle
- `angle_arc` — a small arc showing the angle θ
- `x_axis_projection` — a dashed horizontal line (the cosine component)
- `y_axis_projection` — a dashed vertical line (the sine component)
- `coordinate_axes` — standard XYZ axes with labels
- `function_graph` — a parametric curve in 2D
- `tangent_line` — a short line tangent to a curve at a point
- `point_on_curve` — a sphere marking a specific point
- `area_under_curve` — a filled region
- `number_line` — a horizontal axis with tick marks
- `matrix_grid` — a 2D grid showing matrix transformation effect
- `eigenvector_arrow` — a colored arrow showing an eigenvector direction
- `gradient_arrow` — a vector arrow at a point showing gradient direction
- `normal_vector` — a perpendicular vector to a surface

File: `llm_dsl_translator/motion_verb_map.rs`

Purpose

`MotionVerbMap` maps natural motion verb strings (what the LLM uses) to DSL motion type configurations and required/optional parameters.

What it contains

`VerbDefinition` struct:

- `verb: String` — the verb name the LLM uses.
- `dsl_motion_type: String` — the value for `motion.type` in DSL: `"rotation"`, `"translation"`, `"scale"`, `"trajectory_follow"`, `"parametric_path"`, `"opacity_fade"`, `"color_transition"`, `"molecular_vibration"`, `"oscillate"`.
- `required_parameters: Vec<ParameterSpec>`
- `optional_parameters: Vec<ParameterSpec>`
- `compound_type: Option<String>` — if this verb requires a `compound_motion` block, the compound type: `"parallel"` or `"sequential"`.

ParameterSpec struct:

- `name: String` — the key in `MotionIntent.parameters`.
- `value_type: ParameterType` — `Number`, `Vector3`, `String`, `Bool`, `NumberList`.
- `dsl_field_name: String` — the corresponding field name in the DSL motion block.
- `description: String` — shown to the LLM in prompts.

MotionVerbMap struct:

- `verbs: HashMap<String, VerbDefinition>`

Methods:

- `MotionVerbMap::builtin() -> Self`
- `get(&self, verb: &str) -> Option<&VerbDefinition>`
- `fuzzy_match(&self, verb: &str) -> Vec<&str>`
- `as_json(&self) -> serde_json::Value`

Builtin verbs (minimum set)

- `rotate` → `type: rotation` | required: `axis [f64;3]`, `speed f64` | optional: `from_angle f64`, `to_angle f64`
- `translate` → `type: translation` | required: `direction [f64;3]`, `speed f64`
- `sweep_to` → `type: trajectory_follow` | required: `target_position [f64;3]`, `duration f64`
- `scale_to` → `type: scale` | required: `target_scale [f64;3]`, `duration f64`
- `fade_in` → `opacity_fade` | required: `duration f64` | optional: `from_opacity f64` (default 0)
- `fade_out` → `opacity_fade` | required: `duration f64` | optional: `to_opacity f64` (default 0)
- `oscillate` → `type: molecular_vibration` | required: `frequency f64`, `amplitude f64`
- `draw` → `type: parametric_path` | required: `from f64`, `to f64`, `duration f64`
- `pulse` → compound parallel of `scale_to` (1.2×) and `scale_to` (1.0×) | required: `duration f64`
- `appear` → compound of `fade_in` and optional `sweep_to` | required: `duration f64`

File: `llm_dsl_translator/dsl_builder.rs`

Purpose

`DslBuilder` programmatically constructs valid DSL source text from an `AnimationIntent`. It is the heart of the translator — the code that produces the actual DSL string.

What it contains

`DslBuilder` struct:

- `entity_registry: Arc<EntityRoleRegistry>`
- `verb_map: Arc<MotionVerbMap>`
- `id_generator: IdGenerator`
- `output: String` — the DSL text being assembled, built in mandatory block order.

`DslBuilder::build(intent: &AnimationIntent) -> Result<String, Vec<TranslationError>>` — the primary method. Calls the following private methods in mandatory DSL block order:

1. **`write_scene_block`** — writes the `scene { }` block. Scene name is derived from `intent.concept_id + "." + intent.section_id`. Version is `2`, `ir_version` is `"0.2.0"`, `unit_system` is `"SI"`, `domain` is `"math"`.
2. **`write_library_imports`** — determines which libraries to import based on what entity kinds and motion types are used. Always includes `math: "core_mechanics"`. Adds `math: "vector_calculus"` if any vector field entities are present. Adds `math: "parametric_curves"` if any `draw` motions are present. Adds `geometry: "basic_solids"` if any standard solid geometry is used.
3. **`write_materials`** — generates a `materials { }` block. One `material` entry per unique color used across all entities, named `"mat_{hex_color}"`.
4. **`write_entities`** — for each `EntityIntent`:
   - Looks up the template in `EntityRoleRegistry`.
   - Applies any overrides from the intent (`initial_position`, `initial_scale`, `color_override`).
   - Writes a complete `entity {entity_id} { kind: ... components { ... } }` block.
   - If `entity_intent.represents_variable` is set, generates a `variable_display` sub-block.
5. **`write_constraints`** — currently writes an empty `// no constraints` comment block. Constraint generation is a future enhancement.
6. **`write_motions`** — for each `MotionIntent`:
   - Looks up the `VerbDefinition` in `MotionVerbMap`.
   - Validates all required parameters are present with correct types. Accumulates `TranslationError` objects for any failures (does not abort immediately — collects all errors to return together).
   - Writes the `motion {motion_id} { type: ... target: ... ... }` block with all parameters.
7. **`write_compound_motions`** — for any `MotionIntent.compound_with` groups, generates a `compound_motion { }` block grouping those motions.
8. **`write_trajectories`** — empty for now (trajectory-based motions are a future enhancement).

9. **write_timelines** — generates a `timeline main { }` block. Places each motion as an `event { motion: ... start: ... duration: ... }`. Validates that no two events for the same entity overlap (timing conflict check).
10. **write_annotations** — generates the `annotations { }` block from `intent.annotations`.
11. **write_highlight_schedule** — generates the `highlight_schedule { }` block from `intent.highlights`, converting `at_step` indices to timeline timestamps by mapping through the timeline events generated in step 9.
12. **write_concept_ref** — generates `concept_ref { concept_id: "..." section_id: "..." step_index: N }`.

Error handling: The builder collects all errors during generation and returns them together rather than aborting on the first error. This gives the `FeedbackLoop` a complete list of problems to include in the correction prompt.

File: `llm_dsl_translator/id_generator.rs`

Purpose

`IdGenerator` produces deterministic, unique, valid DSL identifier strings.

What it contains

`IdGenerator` struct:

- `counters: HashMap<String, usize>` — one counter per prefix.

`IdGenerator::motion_id(&mut self, verb: &str, target: &str) -> String` — produces `"{verb}_{target}_{n}"`, e.g. `"rotate_radius_vec_1"`.

`IdGenerator::entity_id(&mut self, role: &str, name: &str) -> String` — uses the provided `entity_intent.entity_id` if it is a valid DSL identifier (only `[a-zA-Z0-9_]` characters, does not start with a digit). If invalid, sanitizes it by replacing invalid characters with `_` and prepending `"e_"` if it starts with a digit.

`IdGenerator::compound_id(&mut self, motions: &[&str]) -> String` — produces `"compound_{hash}"` where `{hash}` is the first 6 chars of the SHA-256 of the concatenated motion IDs.

`is_valid_dsl_identifier(s: &str) -> bool` — validates per DSL grammar rules.

File: `llm_dsl_translator/translation_validator.rs`

Purpose

`TranslationValidator` runs in two phases: first it validates the `AnimationIntent` structurally (before building DSL), then it validates the generated DSL by running it through the existing `dsl::Compiler`. This is the self-correcting feedback gate.

What it contains

`TranslationValidator` struct:

- `entity_registry: Arc<EntityRoleRegistry>`
- `verb_map: Arc<MotionVerbMap>`
- `compiler: dsl_compiler::Compiler` — the existing DSL compiler from `dsl/lib.rs`.

`validate_intent(intent: &AnimationIntent) -> Vec<TranslationError>` — intent-level validation before DSL generation:

- Checks all entity roles exist in registry.
- Checks all motion verbs exist in verb map.
- Checks all required parameters are present for each verb.
- Checks all `AnnotationIntent.anchor_entity_id` values refer to entities in `intent.entities`.
- Checks all `HighlightIntent.entity_id` values refer to entities in `intent.entities`.
- Checks color indices are 0–15.
- Checks `at_step` values in `HighlightIntent` are plausible (within the motion timeline range).
- Returns an empty `Vec` if all checks pass.

`validate_dsl(dsl_source: &str) -> Vec<TranslationError>` — DSL-level validation after generation:

- Calls `self.compiler.compile(dsl_source, PathBuf::from("generated.dsl"))`.
- On success: returns empty `Vec`.
- On failure: converts each `DslError` from the compiler into a `TranslationError::DslCompileError`.

`validate_full(intent: &AnimationIntent, dsl_source: &str) -> Vec<TranslationError>` — runs both phases in sequence. Returns the union of all errors.

File: `llm_dsl_translator/mod.rs`

Purpose

`Translator` — the public-facing struct.

What it contains

`Translator` struct:

- `entity_registry: Arc<EntityRoleRegistry>`
- `verb_map: Arc<MotionVerbMap>`
- `validator: TranslationValidator`

`Translator::new() -> Self` — initializes with builtin registry and verb map.

`Translator::translate(intent: &AnimationIntent) -> TranslationResult`:

1. `validate_intent()` — returns early with errors if intent is structurally invalid.
2. `DslBuilder::build(intent)` — generates DSL. If builder returns errors, returns them.
3. `validate_dsl(dsl_source)` — runs the full compiler. If errors, returns them.
4. On success: returns `TranslationResult::Success { dsl_source, ir_scene }`.

`TranslationResult` enum:

- `Success { dsl_source: String, ir_scene: IrScene }` — the generated DSL text and the compiled IR (ready for the `SceneLoader`).
- `Failure { errors: Vec<TranslationError> }` — returned to `FeedbackLoop`.

Dependencies Summary

Depends on	Why
`llm_orchestrator/animation_intent.rs` (via `From` conversion)	Receives `AnimationIntent`
`dsl/lib.rs` (`Compiler`)	Runs the full DSL compiler on generated DSL for validation
`dsl/lower_to_ir.rs` (`IrScene`)	Returns the compiled IR on success
Standard library, `serde_json`	Parameter handling, JSON serialization for prompt embedding
`sha2`	Compound motion ID generation

Consumed by

Consumer	What it reads
`llm_orchestrator/feedback_loop.rs`	`TranslationValidator::validate_intent()` for pre-generation checks; `TranslationResult::Failure` errors for correction prompts
`server/session.rs`	`TranslationResult::Success { ir_scene }` — loads the IR directly into the runtime